```sql
-- Module 4 – SQL for Data Analytics (M4-A1)
-- MySQL 8.0+
-- Complete sample solution for Sections A, B, C and D.
-- Run in a test database. Sample data is illustrative.

CREATE DATABASE IF NOT EXISTS food_delivery_analytics;
USE food_delivery_analytics;

-- ============================================================
-- SECTION A — CONCEPT APPLICATION (SQL examples)
-- ============================================================

-- A1: GROUP BY + HAVING: last month's restaurant revenue and order count.
-- This assumes order_date and total_amount are in delivery_orders.
SELECT restaurant_name,
       SUM(total_amount) AS total_revenue,
       COUNT(*) AS total_orders
FROM delivery_orders
WHERE order_date >= DATE_FORMAT(CURRENT_DATE - INTERVAL 1 MONTH,
'%Y-%m-01')
  AND order_date <  DATE_FORMAT(CURRENT_DATE, '%Y-%m-01')
GROUP BY restaurant_name
HAVING SUM(total_amount) > 100000
   AND COUNT(*) >= 50;

-- A2: Keep every customer, even customers with no orders.
SELECT c.customer_id, c.name, o.order_id, o.order_date, o.amount
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.customer_id;

-- Most recent order per customer (MySQL 8+).
WITH ranked_orders AS (
    SELECT o.*,
           ROW_NUMBER() OVER (
              PARTITION BY o.customer_id
              ORDER BY o.order_date DESC, o.order_id DESC
           ) AS rn
    FROM orders AS o
)
SELECT c.customer_id, c.name, ro.order_id, ro.order_date, ro.amount
FROM customers AS c
LEFT JOIN ranked_orders AS ro
  ON ro.customer_id = c.customer_id AND ro.rn = 1;
```

```sql
-- A3: Scalar subquery compares each agent's average to platform average.
SELECT agent_name, AVG(delivery_time_mins) AS avg_delivery_time
FROM deliveries
GROUP BY agent_name
HAVING AVG(delivery_time_mins) < (
   SELECT AVG(delivery_time_mins) FROM deliveries
);

-- A4: RANK gives tied ranks with gaps; DENSE_RANK has no gaps.
WITH restaurant_revenue AS (
   SELECT city, restaurant_name, SUM(total_amount) AS revenue
   FROM delivery_orders
   GROUP BY city, restaurant_name
)
SELECT city, restaurant_name, revenue,
      RANK() OVER (PARTITION BY city ORDER BY revenue DESC) AS rank_with_gaps,
      DENSE_RANK() OVER (PARTITION BY city ORDER BY revenue DESC) AS
dense_rank_no_gaps
FROM restaurant_revenue;

-- A5: CASE order matters. Exactly 5000 is Gold with this condition order.
SELECT customer_id, total_spend,
      CASE
         WHEN total_spend > 5000 THEN 'Platinum'
         WHEN total_spend >= 2000 THEN 'Gold'
         ELSE 'Standard'
      END AS customer_tier
FROM customer_spending;

-- A6: Index example; create only after checking the query plan.
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
-- Indexes can speed up JOIN/filter lookups, but add storage and slow writes.

-- ============================================================
-- SECTION B — PRACTICAL CODING TASKS
-- ============================================================

-- TASK 1: Menu & Order Explorer
CREATE TABLE IF NOT EXISTS food_orders (
   order_id INT PRIMARY KEY,
   customer_name VARCHAR(100) NOT NULL,
   restaurant_name VARCHAR(100) NOT NULL,
```

```
    item_ordered VARCHAR(100) NOT NULL,
    quantity INT NOT NULL,
    price_per_item DECIMAL(10,2) NOT NULL,
    order_date DATE NOT NULL,
    city VARCHAR(60) NOT NULL
);

INSERT INTO food_orders VALUES
(1,'Aarav Shah','Spice Route','Paneer Tikka',2,320.00,'2026-10-20','Mumbai'),
(2,'Diya Patel','Green Bowl','Salad Bowl',1,240.00,'2026-10-21','Pune'),
(3,'Kabir Mehta','Bombay Biryani','Chicken Biryani',3,280.00,'2026-10-22','Mumbai'),
(4,'Anaya Desai','Dosa Corner','Masala Dosa',4,120.00,'2026-10-23','Surat'),
(5,'Ishaan Rao','Spice Route','Butter Paneer',2,410.00,'2026-10-24','Mumbai'),
(6,'Meera Joshi','Taco Town','Tacos',3,190.00,'2026-10-24','Pune'),
(7,'Rohan Shah','Bombay Biryani','Veg Biryani',2,220.00,'2026-10-25','Mumbai'),
(8,'Sara Khan','Coastal Curry','Fish Curry',1,550.00,'2026-10-26','Surat');

-- All Mumbai orders, most expensive item first.
SELECT *
FROM food_orders
WHERE city = 'Mumbai'
ORDER BY price_per_item DESC;

-- All distinct restaurants with at least one order.
SELECT DISTINCT restaurant_name
FROM food_orders
ORDER BY restaurant_name;

-- Top 5 most expensive individual orders (total line value = quantity * unit price).
SELECT order_id, customer_name, restaurant_name, item_ordered,
       quantity, price_per_item,
       quantity * price_per_item AS individual_order_value
FROM food_orders
ORDER BY individual_order_value DESC
LIMIT 5;

-- TASK 2: Restaurant Revenue Summary
CREATE TABLE IF NOT EXISTS delivery_orders (
    order_id INT PRIMARY KEY,
    restaurant_name VARCHAR(100) NOT NULL,
    city VARCHAR(60) NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    order_status VARCHAR(30) NOT NULL,
```

```
    order_date DATE NOT NULL
);

INSERT INTO delivery_orders VALUES
(101,'Spice Route','Mumbai',1800.00,'Completed','2026-10-01'),
(102,'Spice Route','Mumbai',2200.00,'Completed','2026-10-02'),
(103,'Spice Route','Mumbai',1600.00,'Completed','2026-10-03'),
(104,'Spice Route','Mumbai',2500.00,'Completed','2026-10-04'),
(105,'Green Bowl','Pune',1200.00,'Completed','2026-10-01'),
(106,'Green Bowl','Pune',900.00,'Completed','2026-10-02'),
(107,'Green Bowl','Pune',1400.00,'Completed','2026-10-03'),
(108,'Bombay Biryani','Mumbai',3200.00,'Completed','2026-10-01'),
(109,'Bombay Biryani','Mumbai',2800.00,'Completed','2026-10-02'),
(110,'Dosa Corner','Surat',700.00,'Completed','2026-10-01'),
(111,'Dosa Corner','Surat',850.00,'Cancelled','2026-10-02'),
(112,'Coastal Curry','Surat',4200.00,'Completed','2026-10-01');

SELECT restaurant_name,
       SUM(total_amount) AS total_revenue,
       COUNT(*) AS total_orders,
       AVG(total_amount) AS average_order_value
FROM delivery_orders
WHERE order_status = 'Completed'
GROUP BY restaurant_name
HAVING SUM(total_amount) > 5000;

SELECT restaurant_name,
       SUM(total_amount) AS total_revenue,
       COUNT(*) AS total_orders,
       AVG(total_amount) AS average_order_value,
       CASE
          WHEN SUM(total_amount) > 10000 THEN 'High Revenue'
          ELSE 'Moderate Revenue'
       END AS revenue_category
FROM delivery_orders
WHERE order_status = 'Completed'
GROUP BY restaurant_name
HAVING SUM(total_amount) > 5000;

-- TASK 3: Customer–Order–Restaurant Joined Report
CREATE TABLE IF NOT EXISTS customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
```

```
    city VARCHAR(60) NOT NULL,
    phone VARCHAR(20)
);

CREATE TABLE IF NOT EXISTS restaurants (
    restaurant_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    cuisine_type VARCHAR(60) NOT NULL,
    city VARCHAR(60) NOT NULL
);

CREATE TABLE IF NOT EXISTS orders (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    restaurant_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    order_date DATE NOT NULL,
    CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id) REFERENCES
customers(customer_id),
    CONSTRAINT fk_orders_restaurant FOREIGN KEY (restaurant_id) REFERENCES
restaurants(restaurant_id)
);

INSERT INTO customers VALUES
(1,'Aarav Shah','Mumbai','9000000001'),
(2,'Diya Patel','Pune','9000000002'),
(3,'Kabir Mehta','Mumbai','9000000003'),
(4,'Anaya Desai','Surat','9000000004'),
(5,'Ishaan Rao','Pune','9000000005'),
(6,'Meera Joshi','Surat','9000000006');

INSERT INTO restaurants VALUES
(1,'Spice Route','Indian','Mumbai'),
(2,'Green Bowl','Healthy','Pune'),
(3,'Bombay Biryani','Mughlai','Mumbai'),
(4,'Dosa Corner','South Indian','Surat'),
(5,'Coastal Curry','Seafood','Surat'),
(6,'Taco Town','Mexican','Pune');

INSERT INTO orders VALUES
(201,1,1,1250.00,'2026-10-20'),
(202,2,2,850.00,'2026-10-20'),
(203,3,3,1450.00,'2026-10-21'),
```

```
(204,1,3,950.00,'2026-10-22'),
(205,4,4,620.00,'2026-10-23'),
(206,5,6,1100.00,'2026-10-24');

-- Each order with customer and restaurant.
SELECT o.order_id, c.name AS customer_name,
       r.name AS restaurant_name, o.amount, o.order_date
FROM orders AS o
INNER JOIN customers AS c ON c.customer_id = o.customer_id
INNER JOIN restaurants AS r ON r.restaurant_id = o.restaurant_id;

-- Every customer, including those with no orders.
SELECT c.customer_id, c.name AS customer_name, c.city,
       o.order_id, o.amount, o.order_date,
       r.name AS restaurant_name
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.customer_id
LEFT JOIN restaurants AS r ON r.restaurant_id = o.restaurant_id
ORDER BY c.customer_id, o.order_date;

-- Orders where customer and restaurant are in the same city.
SELECT o.order_id, c.name AS customer_name, r.name AS restaurant_name,
       c.city, o.amount, o.order_date
FROM orders AS o
INNER JOIN customers AS c ON c.customer_id = o.customer_id
INNER JOIN restaurants AS r ON r.restaurant_id = o.restaurant_id
WHERE c.city = r.city;

-- TASK 4: Delivery Agent Performance Ranking
CREATE TABLE IF NOT EXISTS deliveries (
   delivery_id INT PRIMARY KEY,
   agent_name VARCHAR(100) NOT NULL,
   restaurant_name VARCHAR(100) NOT NULL,
   delivery_time_mins INT NULL,
   order_date DATE NOT NULL
);

INSERT INTO deliveries VALUES
(301,'Asha','Spice Route',24,'2026-10-20'),
(302,'Asha','Green Bowl',28,'2026-10-21'),
(303,'Asha','Dosa Corner',25,'2026-10-22'),
(304,'Bharat','Spice Route',35,'2026-10-20'),
(305,'Bharat','Taco Town',38,'2026-10-21'),
```

```
(306,'Bharat','Green Bowl',32,'2026-10-22'),
(307,'Charu','Coastal Curry',21,'2026-10-20'),
(308,'Charu','Dosa Corner',23,'2026-10-21'),
(309,'Dev','Bombay Biryani',42,'2026-10-20'),
(310,'Dev','Spice Route',40,'2026-10-22'),
(311,'Dev','Taco Town',NULL,'2026-10-23'),
(312,'Esha','Green Bowl',29,'2026-10-24');

WITH agent_stats AS (
    SELECT agent_name,
           AVG(delivery_time_mins) AS avg_delivery_time,
           COUNT(delivery_time_mins) AS timed_deliveries
    FROM deliveries
    GROUP BY agent_name
),
ranked_agents AS (
    SELECT agent_name, avg_delivery_time, timed_deliveries,
           RANK() OVER (ORDER BY avg_delivery_time ASC) AS speed_rank
    FROM agent_stats
)
SELECT agent_name, avg_delivery_time, timed_deliveries, speed_rank,
       (SELECT AVG(delivery_time_mins) FROM deliveries) AS platform_avg,
       CASE
           WHEN avg_delivery_time <= (SELECT AVG(delivery_time_mins) FROM deliveries)
               THEN 'On Track'
           ELSE 'Needs Improvement'
       END AS performance_status
FROM ranked_agents
ORDER BY speed_rank, agent_name;

-- ============================================================
-- SECTION C — MINI CAPSTONE PROJECT
-- Uses customers, restaurants, orders, delivery_agents.
-- This section uses separate table names (capstone_*) to avoid
-- conflicts with Section B tables.
-- ============================================================

CREATE TABLE IF NOT EXISTS capstone_customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    city VARCHAR(60) NOT NULL,
    phone VARCHAR(20)
);
```

```
CREATE TABLE IF NOT EXISTS capstone_restaurants (
   restaurant_id INT PRIMARY KEY,
   restaurant_name VARCHAR(100) NOT NULL,
   cuisine_type VARCHAR(60) NOT NULL,
   city VARCHAR(60) NOT NULL
);

CREATE TABLE IF NOT EXISTS delivery_agents (
   agent_id INT PRIMARY KEY,
   agent_name VARCHAR(100) NOT NULL,
   home_city VARCHAR(60) NOT NULL
);

CREATE TABLE IF NOT EXISTS capstone_orders (
   order_id INT PRIMARY KEY,
   customer_id INT NOT NULL,
   restaurant_id INT NOT NULL,
   agent_id INT NOT NULL,
   order_amount DECIMAL(10,2) NOT NULL,
   order_date DATE NOT NULL,
   delivery_time_mins INT NULL,
   order_status VARCHAR(30) NOT NULL DEFAULT 'Completed',
   CONSTRAINT fk_cap_orders_customer FOREIGN KEY (customer_id) REFERENCES
capstone_customers(customer_id),
   CONSTRAINT fk_cap_orders_restaurant FOREIGN KEY (restaurant_id) REFERENCES
capstone_restaurants(restaurant_id),
   CONSTRAINT fk_cap_orders_agent FOREIGN KEY (agent_id) REFERENCES
delivery_agents(agent_id)
);

INSERT INTO capstone_customers VALUES
(1,'Aarav Shah','Mumbai','9100000001'),
(2,'Diya Patel','Pune','9100000002'),
(3,'Kabir Mehta','Surat','9100000003'),
(4,'Anaya Desai','Mumbai','9100000004'),
(5,'Ishaan Rao','Pune','9100000005'),
(6,'Meera Joshi','Surat','9100000006'),
(7,'Rohan Shah','Mumbai','9100000007'),
(8,'Sara Khan','Pune','9100000008'),
(9,'Vivaan Joshi','Surat','9100000009'),
(10,'Kiara Mehta','Mumbai','9100000010');
```

```
INSERT INTO capstone_restaurants VALUES
(1,'Spice Route','Indian','Mumbai'),
(2,'Green Bowl','Healthy','Pune'),
(3,'Bombay Biryani','Mughlai','Mumbai'),
(4,'Dosa Corner','South Indian','Surat'),
(5,'Coastal Curry','Seafood','Surat'),
(6,'Taco Town','Mexican','Pune'),
(7,'Wok Express','Chinese','Mumbai'),
(8,'Pizza Planet','Italian','Pune'),
(9,'Curry Leaf','Indian','Surat'),
(10,'Urban Tandoor','North Indian','Mumbai');

INSERT INTO delivery_agents VALUES
(1,'Asha','Mumbai'),
(2,'Bharat','Pune'),
(3,'Charu','Surat'),
(4,'Dev','Mumbai'),
(5,'Esha','Pune'),
(6,'Farhan','Surat'),
(7,'Gita','Mumbai'),
(8,'Harsh','Pune'),
(9,'Ira','Surat'),
(10,'Jay','Mumbai');

INSERT INTO capstone_orders VALUES
(401,1,1,1,1800.00,'2026-10-21',24,'Completed'),
(402,2,2,2,2200.00,'2026-10-21',32,'Completed'),
(403,3,4,3,1200.00,'2026-10-22',27,'Completed'),
(404,4,1,4,1600.00,'2026-10-22',29,'Completed'),
(405,5,2,5,2400.00,'2026-10-23',30,'Completed'),
(406,6,5,6,3200.00,'2026-10-23',26,'Completed'),
(407,7,3,1,2800.00,'2026-10-24',25,'Completed'),
(408,8,6,2,1700.00,'2026-10-24',34,'Completed'),
(409,9,9,3,900.00,'2026-10-25',31,'Completed'),
(410,10,7,4,5300.00,'2026-10-25',22,'Completed'),
(411,1,1,1,2100.00,'2026-10-26',23,'Completed'),
(412,2,2,2,1900.00,'2026-10-26',33,'Completed'),
(413,3,4,3,2300.00,'2026-10-26',28,'Completed'),
(414,4,1,4,2600.00,'2026-10-27',27,'Completed'),
(415,5,2,5,1600.00,'2026-10-27',29,'Completed'),
(416,6,5,6,1400.00,'2026-10-27',25,'Completed'),
(417,7,3,1,1900.00,'2026-10-27',24,'Completed'),
(418,8,6,2,800.00,'2026-10-27',35,'Completed'),
```

```
(419,9,9,3,1500.00,'2026-10-27',30,'Completed'),
(420,10,7,4,2200.00,'2026-10-27',21,'Completed');

-- C1: Revenue per restaurant; only restaurants with more than 3 orders.
SELECT r.restaurant_id, r.restaurant_name, r.city,
       SUM(o.order_amount) AS total_revenue,
       COUNT(*) AS total_orders,
       AVG(o.order_amount) AS average_order_value
FROM capstone_restaurants AS r
JOIN capstone_orders AS o ON o.restaurant_id = r.restaurant_id
WHERE o.order_status = 'Completed'
GROUP BY r.restaurant_id, r.restaurant_name, r.city
HAVING COUNT(*) > 3
ORDER BY total_revenue DESC;

-- C2: Customer segmentation by lifetime spend.
SELECT c.customer_id, c.customer_name,
       COALESCE(SUM(o.order_amount), 0) AS total_spend,
       CASE
          WHEN COALESCE(SUM(o.order_amount), 0) > 5000 THEN 'Platinum'
          WHEN COALESCE(SUM(o.order_amount), 0) >= 2000 THEN 'Gold'
          ELSE 'Standard'
       END AS customer_tier
FROM capstone_customers AS c
LEFT JOIN capstone_orders AS o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_spend DESC;

-- C3: Top 3 restaurants by revenue in each city, using DENSE_RANK.
WITH restaurant_city_revenue AS (
   SELECT r.city, r.restaurant_id, r.restaurant_name,
          SUM(o.order_amount) AS total_revenue
   FROM capstone_restaurants AS r
   JOIN capstone_orders AS o ON o.restaurant_id = r.restaurant_id
   WHERE o.order_status = 'Completed'
   GROUP BY r.city, r.restaurant_id, r.restaurant_name
),
ranked_restaurants AS (
   SELECT *,
          DENSE_RANK() OVER (
             PARTITION BY city ORDER BY total_revenue DESC
          ) AS revenue_rank
   FROM restaurant_city_revenue
```

```
)
SELECT city, restaurant_name, total_revenue, revenue_rank
FROM ranked_restaurants
WHERE revenue_rank <= 3
ORDER BY city, revenue_rank, restaurant_name;

-- C4: Most recent order per customer, using ROW_NUMBER.
WITH customer_recent_orders AS (
    SELECT o.*,
           ROW_NUMBER() OVER (
              PARTITION BY customer_id
              ORDER BY order_date DESC, order_id DESC
           ) AS rn
    FROM capstone_orders AS o
)
SELECT c.customer_id, c.customer_name, cro.order_id,
       cro.order_date, cro.order_amount, cro.delivery_time_mins
FROM capstone_customers AS c
LEFT JOIN customer_recent_orders AS cro
  ON cro.customer_id = c.customer_id AND cro.rn = 1
ORDER BY c.customer_id;

-- C5: View joining all four tables.
CREATE OR REPLACE VIEW vw_daily_order_summary AS
SELECT o.order_date,
       c.customer_name,
       r.restaurant_name,
       a.agent_name,
       o.order_amount,
       o.delivery_time_mins
FROM capstone_orders AS o
JOIN capstone_customers AS c ON c.customer_id = o.customer_id
JOIN capstone_restaurants AS r ON r.restaurant_id = o.restaurant_id
JOIN delivery_agents AS a ON a.agent_id = o.agent_id;

-- Query orders from the last 7 days, including today.
SELECT *
FROM vw_daily_order_summary
WHERE order_date >= CURRENT_DATE - INTERVAL 6 DAY
  AND order_date < CURRENT_DATE + INTERVAL 1 DAY
ORDER BY order_date DESC, customer_name;

-- ============================================================
```

```sql
-- SECTION D — AI-AUGMENTED LEARNING
-- Exact prompt used:
-- "Write a MySQL 8 query to find the top 3 delivery agents per city
-- for a food delivery platform, ranked by completed order count.
-- Include each agent's average delivery time. Only include agents
-- with more than 5 completed orders in that city. Sort by city
-- alphabetically and rank ascending. Exclude NULL delivery times
-- from average delivery time without excluding their completed orders."
--
-- D1: Original AI query (intentionally contains a logical bug):
-- It ranks agents before filtering to >5 completed orders.
WITH original_agent_city_stats AS (
    SELECT a.home_city AS city, a.agent_id, a.agent_name,
           COUNT(o.order_id) AS completed_orders,
           AVG(o.delivery_time_mins) AS avg_delivery_time
    FROM delivery_agents AS a
    JOIN capstone_orders AS o ON o.agent_id = a.agent_id
    WHERE o.order_status = 'Completed'
    GROUP BY a.home_city, a.agent_id, a.agent_name
),
original_ranked AS (
    SELECT *,
           RANK() OVER (
              PARTITION BY city ORDER BY completed_orders DESC
           ) AS agent_rank
    FROM original_agent_city_stats
)
SELECT city, agent_name, completed_orders, avg_delivery_time, agent_rank
FROM original_ranked
WHERE completed_orders > 5
  AND agent_rank <= 3
ORDER BY city, agent_rank;

-- D2: Corrected query: apply >5 filter before ranking.
WITH agent_city_stats AS (
    SELECT a.home_city AS city, a.agent_id, a.agent_name,
           COUNT(o.order_id) AS completed_orders,
           AVG(o.delivery_time_mins) AS avg_delivery_time
    FROM delivery_agents AS a
    JOIN capstone_orders AS o ON o.agent_id = a.agent_id
    WHERE o.order_status = 'Completed'
    GROUP BY a.home_city, a.agent_id, a.agent_name
    HAVING COUNT(o.order_id) > 5
```

```
),
ranked_agent_city_stats AS (
    SELECT *,
           RANK() OVER (
               PARTITION BY city ORDER BY completed_orders DESC
           ) AS agent_rank
    FROM agent_city_stats
)
SELECT city, agent_name, completed_orders, avg_delivery_time, agent_rank
FROM ranked_agent_city_stats
WHERE agent_rank <= 3
ORDER BY city ASC, agent_rank ASC;

-- D3: Explanation (include in submission note):
-- The original query filtered completed_orders > 5 after RANK() was
-- calculated, so excluded agents could occupy ranks and leave gaps or
-- produce fewer than three qualifying agents. The corrected query uses
-- HAVING in the stats CTE before the window function ranks agents.
-- AVG ignores NULL delivery times, while COUNT(order_id) still counts
-- completed orders even when delivery_time_mins is NULL.

-- End of solution.
```